



# Introdução aos comandos DDL, DML e DQL

**DDL (Data Definition Language)** é usada para definir e estruturar o banco de dados, incluindo a criação de tabelas, colunas e chaves. Já o **DML (Data Manipulation Language)** é responsável por manipular os dados armazenados, como inserir, atualizar, excluir e consultar informações.

Esses dois grupos de comandos são fundamentais para o gerenciamento de bancos de dados relacionais, permitindo uma definição robusta da estrutura e controle eficiente dos dados.



por Prof. Cíntia Pinho

# Criando o banco de dados

## Sintaxe básica

Para criar um banco de dados, usamos o comando **CREATE DATABASE** seguido do nome desejado. Logo após, deve-se selecionar o banco com **USE**.

Exemplo simples:

```
CREATE DATABASE loja;  
USE loja;
```

## Importância da criação inicial

Definir o banco de dados antes de criar tabelas é essencial para garantir que as estruturas e dados sejam organizados no local correto.

Essa etapa inicial define o contexto para todas as operações futuras.

# Criando tabelas com DDL – CREATE TABLE

## Sintaxe de criação

O comando **CREATE TABLE** define tabelas com colunas, tipos de dados e restrições como chave primária e auto incremento.

Formato básico:

```
CREATE TABLE nome_tabela (nome_coluna tipo  
[restrições], ...);
```

## Exemplo prático

Criação das tabelas categorias e produtos com chaves primárias e tipos compatíveis.

```
CREATE TABLE categorias (
```

```
id_categoria INT PRIMARY KEY AUTO_INCREMENT,  
nome_cat VARCHAR(255)
```

```
);
```

```
CREATE TABLE produtos (
```

```
id_produto INT PRIMARY KEY AUTO_INCREMENT,
```

```
nome VARCHAR(255),
```

```
preco DECIMAL(10,2),
```

```
quant INT,
```

```
marca VARCHAR(255),
```

```
id_categoria INT);
```

# Criando relacionamentos com FOREIGN KEY

## Sintaxe do comando

Usamos **ALTER TABLE** para adicionar chaves estrangeiras, ligando uma coluna da tabela filha a uma chave da tabela pai.

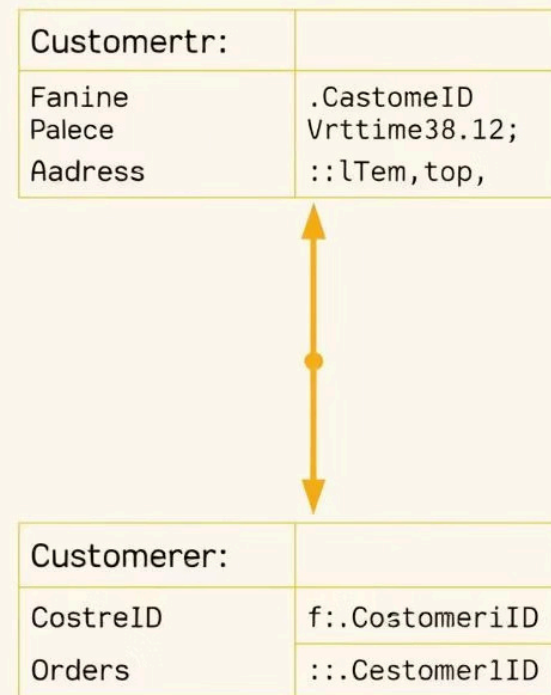
## Exemplo prático

Associar a tabela produtos à categorias para manter integridade referencial:

```
ALTER TABLE produtos ADD FOREIGN KEY (id_categoria)  
REFERENCES categorias(id_categoria);
```

## Benefícios do relacionamento

Garante que produtos só possam pertencer a categorias válidas, evitando dados órfãos e inconsistências no banco.



# Criando Chave Estrangeira na Criação da Tabela

## Definição Direta na Tabela

Inclua a chave estrangeira já na declaração do **CREATE TABLE**.

## Exemplo para Produtos

Adicione **FOREIGN KEY** na criação para referenciar categorias.

## Vantagens

Agiliza o processo e mantém a integridade referencial desde o início.

Sintaxe:

```
CREATE TABLE nome_tabela (
```

```
coluna1 tipo restrições,
```

```
coluna2 tipo,
```

```
...
```

```
FOREIGN KEY (coluna) REFERENCES tabela_pai(coluna_pai) );
```

Exemplo:

```
CREATE TABLE produtos (
```

```
id_produto INT PRIMARY KEY AUTO_INCREMENT,
```

```
nome VARCHAR(255),
```

```
preco DECIMAL(10,2),
```

```
quant INT,
```

```
marca VARCHAR(255),
```

```
id_categoria INT,
```

```
FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria) );
```

# Alterando tabelas com ALTER TABLE

## Renomear uma coluna

Comando para modificar o nome e tipo de uma coluna existente.

```
ALTER TABLE produtos CHANGE quant quantidade INT;
```

## Adicionar e remover colunas

- **Adicionar:**

```
ALTER TABLE produtos ADD data_validade DATE;
```

- **Remover:**

```
ALTER TABLE produtos DROP COLUMN marca;
```

# Renomeando tabelas com RENAME TABLE

## Sintaxe simples

Para mudar o nome de um banco, basta usar:

**RENAME TABLE nome\_antigo TO nome\_novo;**

## Exemplo prático

Renomear a tabela produtos para itens, mantendo a estrutura intacta:

**RENAME TABLE produtos TO itens;**

## Cuidados importantes

Confirme que não há dependências ativas antes da renomeação para evitar erros em consultas ou relacionamentos.

```
rename-(' table table")  
remlamee_id,  
tepnaye-able endayes_to workers!)
```

```
sille worker tables table,
```

```
//e employee_id,  
emplone_id,  
first name id,  
last tname lid,  
last name.
```

```
Copdating/10 employtes.) to workers:
```

|              | table |
|--------------|-------|
| employeid.   |       |
| first name.. |       |
| last name... |       |
| last name... |       |

# Inserindo dados com INSERT

## Sintaxe para inserção

O comando **INSERT INTO** recebe a tabela, colunas e valores para registrar novas linhas.

```
INSERT INTO nome_tabela (colunas) VALUES (valores);
```

## Exemplo real

Inserindo categorias e produtos com múltiplos registros:

```
INSERT INTO categorias (nome_cat) VALUES  
( 'Eletrônicos'), ( 'Roupas');
```

```
INSERT INTO produtos (nome, preco, quantidade, marca,  
id_categoria) VALUES ('Celular X', 1500.00, 10, 'MarcaX', 1);
```



# Atualizando dados com UPDATE



## Sintaxe

Atualiza valores especificados, restringindo linhas pela cláusula WHERE para evitar alterações em excesso.

```
UPDATE nome_tabela SET  
coluna = novo_valor WHERE  
condição;
```

## Exemplo

Modificar preço do produto chamado 'Celular X':

```
UPDATE produtos SET preco =  
1399.00 WHERE nome =  
'Celular X';
```

## Boas Práticas

Use sempre WHERE para limitar o impacto do UPDATE e teste antecipadamente com SELECT.

# Excluindo dados com DELETE

## Sintaxe para exclusão

Remove linhas específicas de uma tabela, definidas pela condição na cláusula WHERE.

**DELETE FROM nome\_tabela WHERE condição;**

## Exemplo prático

Eliminar o produto com id 1:

**DELETE FROM produtos WHERE id\_produto = 1;**

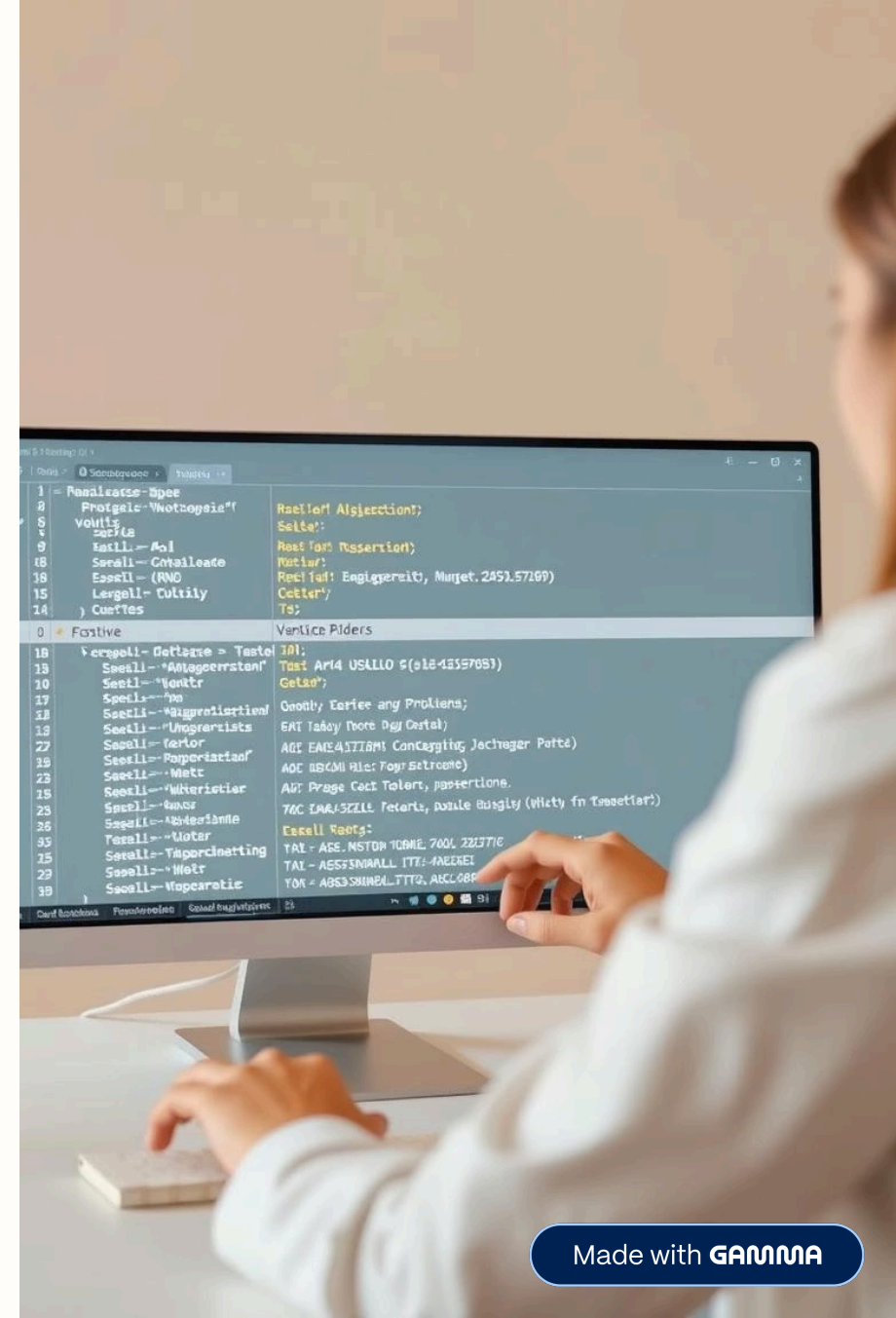
Sem a cláusula WHERE, todos os registros seriam apagados.

# Consultando dados com SELECT e resumo final

| Categoria | Comandos principais               |
|-----------|-----------------------------------|
| DDL       | CREATE, ALTER, RENAME TABLE, DROP |
| DML       | INSERT, UPDATE, DELETE,           |
| DQL       | SELECT                            |

O comando **SELECT** retorna dados da tabela para análise ou uso posterior. Use **SELECT \* FROM tabela;** para extrair todos os registros.

**Dicas finais:** Sempre utilize **WHERE** para limitar alterações em UPDATE e DELETE e teste previamente com SELECT para evitar perda de dados.



O comando `SELECT` faz parte da linguagem DQL (Data Query Language), que é o subconjunto do SQL usado para **consultar dados** em um banco de dados.

Ele é usado sempre que queremos **visualizar informações armazenadas nas tabelas**, como listar produtos, filtrar por marcas, buscar preços, contar registros, entre outras operações de leitura.

# Sintaxe basica

SELECT colunas FROM tabela

[WHERE condição]

[ORDER BY coluna ASC | DESC]

[LIMIT número];

## SELECTs Simples (iniciantes)

1.   Mostrar todos os produtos:

SELECT \* FROM produtos;

1.   Mostrar nome e preço dos produtos:

SELECT nome, preco FROM produtos;

1.   Mostrar todos os produtos da marca "Sadia" ou a "que quiser":

SELECT \* FROM produtos WHERE marca = 'Sadia';

1.   Mostrar produtos com preço maior que 20 reais:

SELECT \* FROM produtos WHERE preco > 20;

1.   Mostrar produtos com quantidade menor ou igual a 60:

SELECT \* FROM produtos WHERE quant <= 60;

## SELECTs com ORDER BY e LIMIT

6. Mostrar os 5 produtos mais baratos:

SELECT nome, preco FROM produtos ORDER BY preco ASC LIMIT 5;

7. Mostrar os 5 produtos mais caros:

SELECT nome, preco FROM produtos ORDER BY preco DESC LIMIT 5;

## SELECTs com funções agregadas

8. Contar o total de produtos cadastrados:

SELECT COUNT(\*) AS total\_produtos FROM produtos;

9. Mostrar o preço médio dos produtos:

SELECT AVG(preco) AS preco\_medio FROM produtos;

10. Mostrar o produto com maior quantidade em estoque:

SELECT nome, quant FROM produtos ORDER BY quant DESC LIMIT 1;

## SELECTs com JOIN

11. Mostrar nome do produto e nome da categoria (usando JOIN):

SELECT p.nome, c.nome\_cat FROM produtos p JOIN categorias c ON p.id\_categoria = c.id\_categoria;

12. Mostrar todos os produtos da categoria "Laticínios":

SELECT p.nome, p.marca, p.preco FROM produtos p JOIN categorias c ON p.id\_categoria = c.id\_categoria WHERE c.nome\_cat = 'Laticínios';

13. Mostrar total de produtos por categoria:

SELECT c.nome\_cat, COUNT(\*) AS total\_produtos FROM produtos p JOIN categorias c ON p.id\_categoria = c.id\_categoria GROUP BY c.nome\_cat;

## esafios (intermediário)

14. Mostrar os produtos com preço entre 10 e 20 reais:

```
SELECT * FROM produtos
WHERE preco BETWEEN 10 AND 20;
```

15. Mostrar produtos cujo nome começa com 'P':

```
SELECT * FROM produtos
WHERE nome LIKE 'P%';
```

SELECT \* FROM produtos WHERE nome LIKE 'P%';

16. Mostrar as marcas distintas cadastradas:

```
SELECT DISTINCT marca FROM produtos;
```